B28: SQL SuperHero Program : Session 34

1 message

Vishal Jaiswal <vishaldbdev@gmail.com> 19 April 2025 at 13:47
Bcc: satishhattekar1997@gmail.com

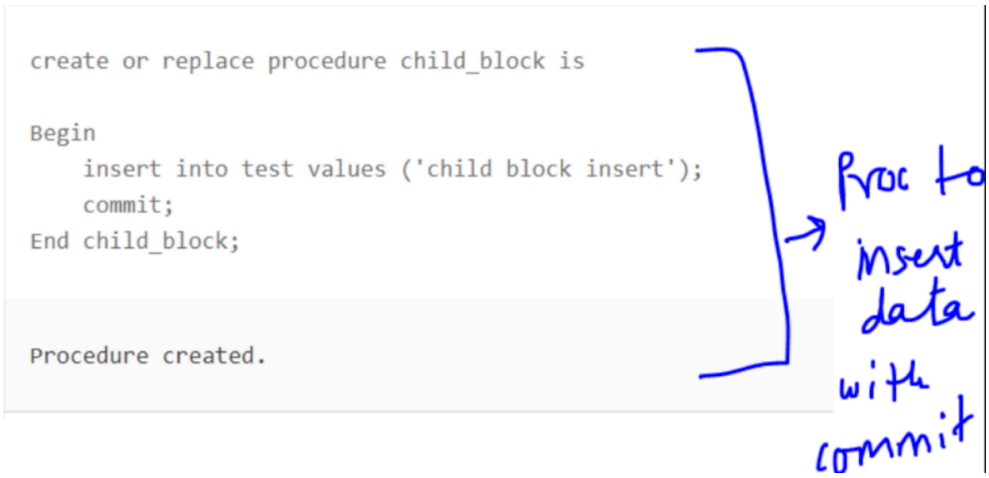
Pragma Autonomous Transaction

Autonomous transactions allow you to create a new transaction within a transaction that may commit. or roll back changes, independently of its parent transaction. The AUTONOMOUS_TRANSACTION pragma changes the way a subprogram works within a transaction. A subprogram marked with this pragma can do SQL operations and commit or roll back those operations, without committing or rolling back the data in the main transaction.
ragmas are processed at compile time, not at run time. They pass information to the compiler.PL/SQL will treat the whole modification as a single transaction and saving/discarding this transaction affects the entire pending changes in that session. Autonomous Transaction provides a functionality to the developer in which it allows to do changes in a separate transaction and to save/discard that particular transaction without affecting the main session transaction.
An autonomous transaction is an independent transaction started by a calling program. A commit or rollback of SQL statements in the autonomous transaction has no effect on the commit or rollback in a transaction of the calling program. A commit or rollback in the calling program has no effect on the commit or rollback of SQL statements in the autonomous transaction.

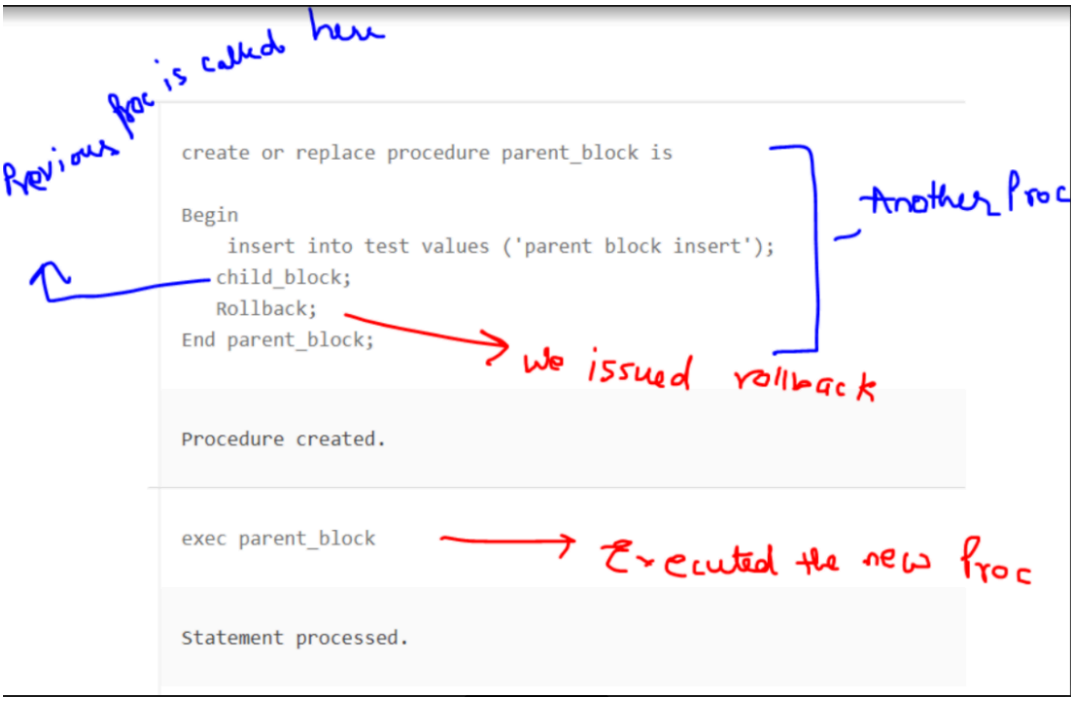
Characteristics:

1. This autonomous transaction can be specified at subprogram level.
2. To make any subprogram to work in a different transaction, the keyword 'PRAGMA AUTONOMOUS_TRANSATION' should be given in the declarative section of that block.
3. It will instruct that compiler to treat this as the separate transaction and saving/discarding inside this block will not reflect in the main transaction.
4. Issuing COMMIT or ROLLBACK is mandatory before going out of this autonomous transaction to the main transaction because at any time only one transaction can be active.
5. So once we make an autonomous transaction we need to save it and complete the transaction then only we can move back to the main transaction.

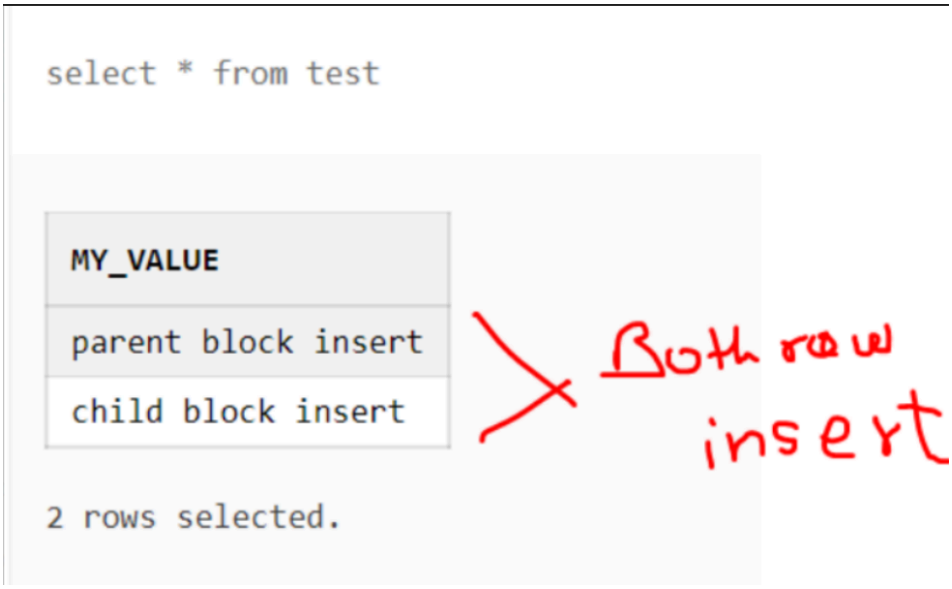
```
create table test(my_value varchar2(20));

create or replace procedure child_block is

Begin
   insert into test values ('child block insert');
   commit;
End child_block;
```



Lets Create one more procedure and we will call the above procedure there;



If you see the Parent_block procedure, here I am inserting a row into test table and just after that call the child_block procedure. In Child Proc, we have commit command issues, so the next rollback command does not matter and it commits the data.



Lets make the child block as Pragma Autonomous. Which allows to run it independent.

```
create or replace procedure child_block is
pragma autonomous_transaction;
Begin
    insert into test values ('child block insert');
    commit;
End child_block;

Procedure created.
```

Added

Pragma is an instruction to the compiler that treats me differently. Now if I execute the parent procedure, it will be totally indelendent of child. Rollback or commit will not impact to each other.

```
exec parent_block

Statement processed.

select * from test
```

MY_VALUE
child block insert

only from child

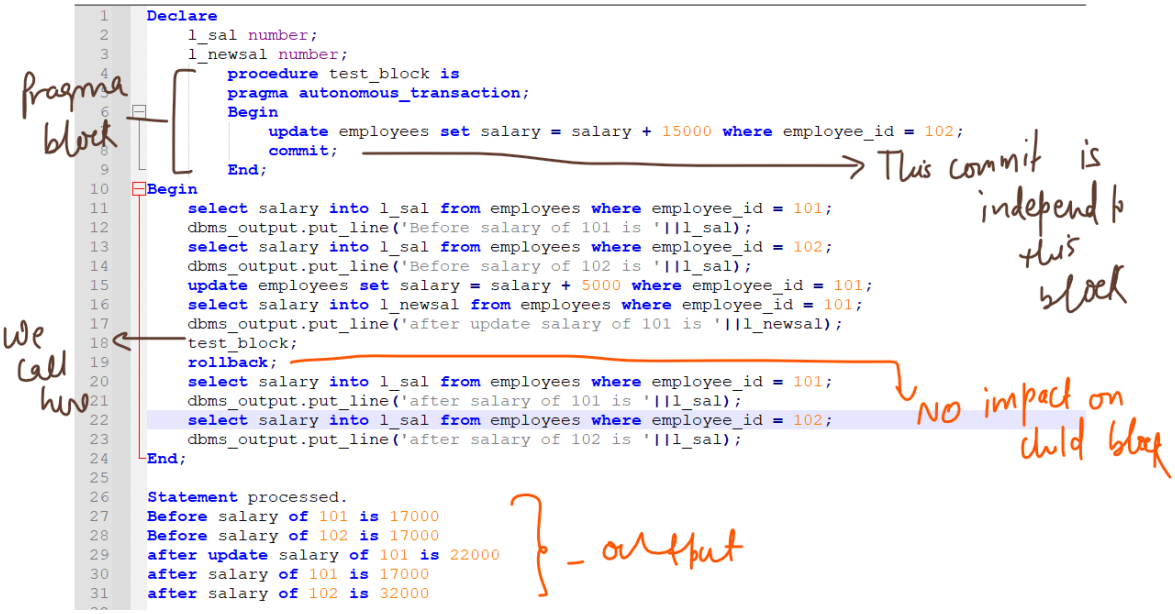
Because parent is written to rollback. See one more example:

Example:

create table employee as select * from hr.employees; -- created a table to test pragma

```
Declare
   l_sal number;
l_newsal number;
procedure test_block is
pragma autonomous_transaction;
      Begin
update employees set salary = salary + 15000 where employee_id = 102;
commit;
      End;
Begin
select salary into l_sal from employees where employee_id = 101;
dbms_output.put_line('Before salary of 101 is '||l_sal);
select salary into l_sal from employees where employee_id = 102;
dbms_output.put_line('Before salary of 102 is '||l_sal);
update employees set salary = salary + 5000 where employee_id = 101;
```

```
select salary into l_newsal from employees where employee_id = 101;
dbms_output.put_line('after update salary of 101 is '||l_newsal);
test_block;
rollback;
select salary into l_sal from employees where employee_id = 101;
dbms_output.put_line('after salary of 101 is '||l_sal);
select salary into l_sal from employees where employee_id = 102;
dbms_output.put_line('after salary of 102 is '||l_sal);
End;
```

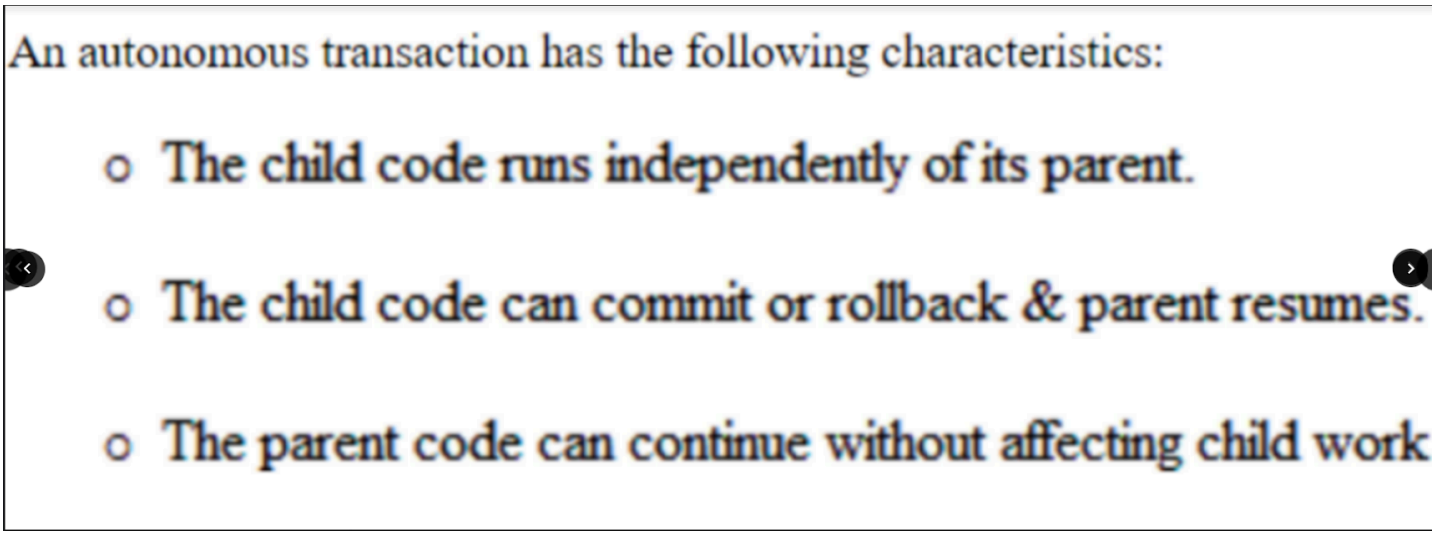


See the above code. In declare section, we declared Pragma as
PRAGMA autonomous_transaction;

- This LIne tells the compiler that this is pragma, treating it independent. Line#5 - Line #11, we declared the pragma code. Now this block is an independent block.
- Once code starts at line # 12, it prints the salary of employee_id 101 and 102 and that is 17000.
- Line#17, we are updating the salary of 101 and increase it by 5000, now salary of employee_id 101 is now 17000+ 5000 = 22000
- Line#21 , prints salary of 101 is 22000. (It is updated)
- Line#22, we call it a pragma transaction. This transaction runs as an independent transaction and update salary of employee_id 102 by 15000. It is committed and now salary of employee_id 102 is 17000+15000 = 32000
- Line#23, we fire rollback. Now we can see here the impact of Pragma. There is no impact of this rollback on pragma code. It is independent, It is committed.
- Line #25, it prints the salary of employee_id 101 as usual 17000 again. It was 22000, but was rolled back.
- Line# 27, no impact of rollback of this and it prints 32000, because it is committed via Autonomous transaction.

Only Use PRAGMA AUTONOMOUS_TRANASCTION For Logging Purposes. Be extremely careful not to abuse this pragma, because you should only use it for logging. If you to need to use an AUTONOMOUS_TRANSACTION pragma aside from logging purposes, then your database design may be flawed.

==========



AUTONOMOUS_TRANSACTION pragma in a function:

```
create table list_of_function_calls
(
   text varchar2(100)
  , ts timestamp
);

create or replace function get_random_number
 return number
is
 pragma autonomous_transaction;
begin
  -- Everything in this function happens in a separate database transaction.
  insert into list_of_function_calls (text, ts)
  values ('I was called!', systimestamp);
  commit;

   -- Number randomly chosen
   return 2;
end get_random_number;
```

it creates a separate transaction from the transaction you are currently working in. Because we included it in our function definition, everything in the body of the function happens in this separate transaction. Therefore, you can commit or rollback within the function, and it will not affect the calling transaction.

We learned that if a function contains DML then we can not call that function in a select statement, but here you can see because of pragma, we can do.

```
21  select get_random_number() from dual;
22
```

GET_RANDOM_NUMBER()
2